Assignment_3_Data_624

Mubashira Qari

2026-02-16

Table of Contents

5.1 Produce forecasts for the following series using whichever of NAIVE(y), SNAIVE(y) or RW(y ~ drift()) is more appropriate in each case:

- Australian Population (`global_economy`)
- Bricks (`aus_production`)
- NSW Lambs (`aus_livestock`)
- Household wealth (`hh_budget`)
- Australian takeaway food turnover (`aus_retail`)

Model Usage

NAIVE(y): Best where the future is simply expected to be the same as the last observed value (common in stock prices).

SNAIVE(y): (Seasonal Naive) Best for data with a clear seasonal pattern (e.g., monthly retail sales). It sets the forecast to be the same as the last observed value from the same season.

RW(y ~ drift()): Best for data that shows a consistent long-term trend (upward or downward) over time.

Australian Population (`global_economy`)

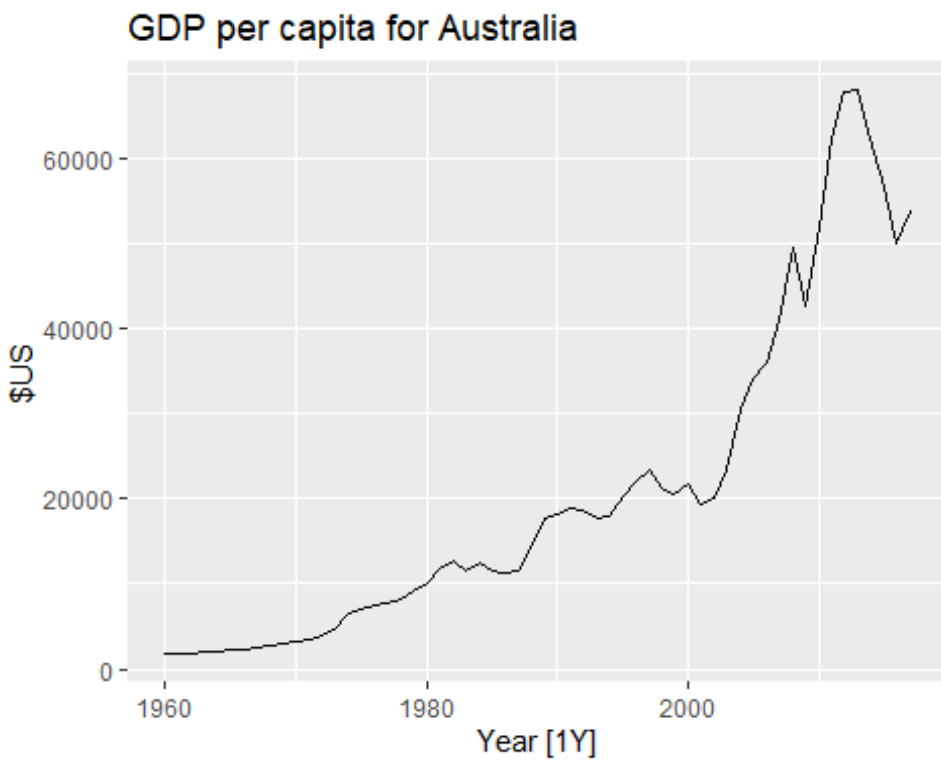
Analyze and Identify the Data Frequency

```
library(fpp3)

## Registered S3 method overwritten by 'tsibble':
##   method               from
##   as_tibble.grouped_df dplyr

## ── Attaching packages ──────────────────────────────────────────── fpp3
1.0.2 ──

## ✓ tibble      3.2.1     ✓ tsibble     1.1.6
## ✓ dplyr       1.1.4     ✓ tsibbledata 0.4.1
## ✓ tidyr       1.3.1     ✓ feasts      0.4.2
```

```
## ✓ lubridate   1.9.4     ✓ fable       0.5.0
## ✓ ggplot2     4.0.0

## Warning: package 'dplyr' was built under R version 4.3.3

## Warning: package 'lubridate' was built under R version 4.3.3

## Warning: package 'tsibble' was built under R version 4.3.3

## Warning: package 'tsibbledata' was built under R version 4.3.3

## ── Conflicts ──────────────────────────────────────────────────
fpp3_conflicts ──
## ✗ lubridate::date()    masks base::date()
## ✗ dplyr::filter()      masks stats::filter()
## ✗ tsibble::intersect() masks base::intersect()
## ✗ tsibble::interval()  masks lubridate::interval()
## ✗ dplyr::lag()         masks stats::lag()
## ✗ tsibble::setdiff()   masks base::setdiff()
## ✗ tsibble::union()     masks base::union()

# global_economy
```

Prepare the Data

```
gdp_pc <- global_economy |>
  mutate(GDP_per_capita = GDP / Population) |>
  filter(!is.na(GDP_per_capita))

view(gdp_pc)

gdp_pc |>
  filter(Country == "Australia") |>
  autoplot(GDP_per_capita) +
  labs(y = "$US", title = "GDP per capita for Australia")
```



Model Selection

To choose between NAIVE(), SNAIVE(), or RW(drift()), we examine the time plot for trend and seasonality.

Trend: The data shows a clear, consistent upward movement.

Seasonality: Seasonality requires multiple observations per year like monthly or quarterly data to see repeating patterns. Therefore, SNAIVE(y) is not applicable.

Growth over Stability: The NAIVE(y) method assumes the future will be exactly like the last observed value, resulting in a flat horizontal line.

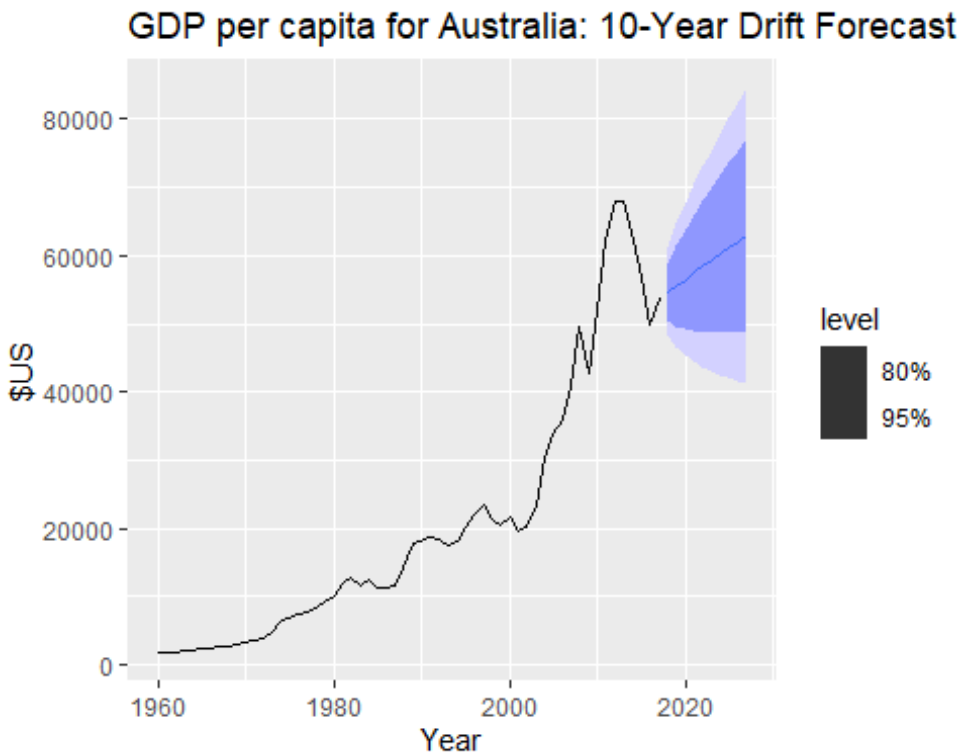
Since the economy typically grows over time, a flat line would likely underestimate the future.

Conclusion: The Drift Method is a variation of the naive approach that forecasts trends by extending the average historical change over time.

Fit the model, Generating Forecast and Plotting

```
# Fit the Drift model
fit_gdp <- gdp_pc |>
  filter(Country == "Australia") |>
  model(Drift = RW(GDP_per_capita ~ drift()))

# Generate a 10-year forecast
fc_gdp <- fit_gdp |>
  forecast(h = "10 years")
```

```
# Plot the forecast alongside historical data
fc_gdp |>
  autoplot(gdp_pc) +
  labs(y = "$US", title = "GDP per capita for Australia: 10-Year Drift
Forecast")
```



Bricks (aus_production)

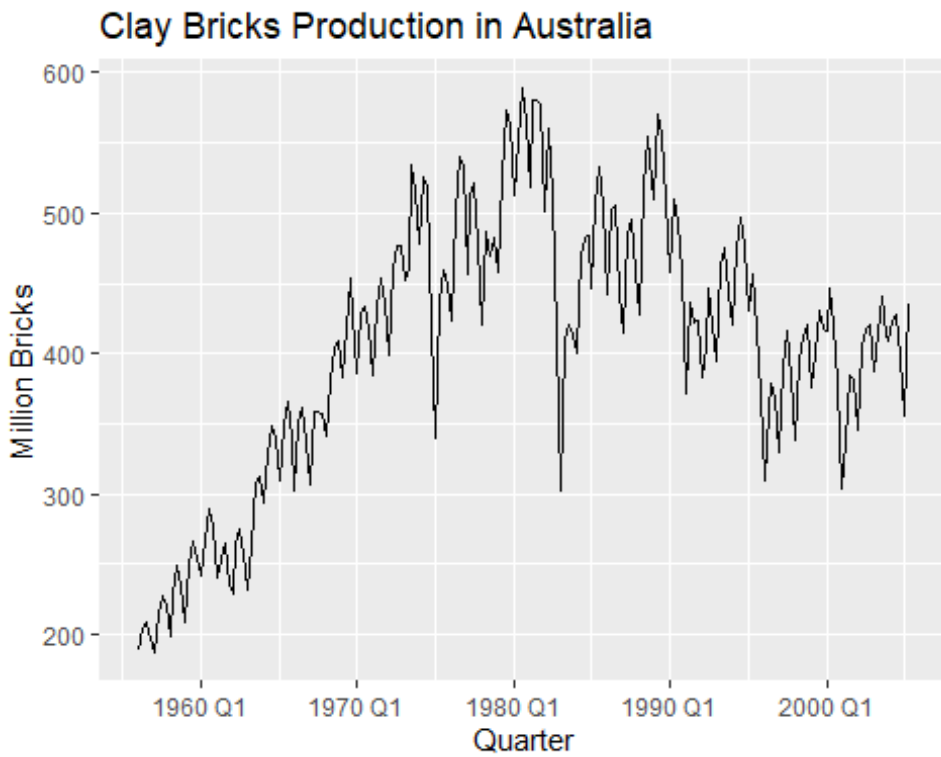
Analyze and Identify the Data Frequency

```
# view(aus_production)
```

Prepare the Data

```
brick_data <- aus_production |>
  filter(!is.na(Bricks))

# view(brick_data)

brick_data <- aus_production |>
  filter(!is.na(Bricks)) |>
  select(Quarter, Bricks)

# brick_data
```

```r
# Create a time plot of the cleaned brick data
brick_data |>
  autoplot(Bricks) +
  labs(title = "Clay Bricks Production in Australia",
       y = "Million Bricks",
       x = "Quarter")
```



Model Selection

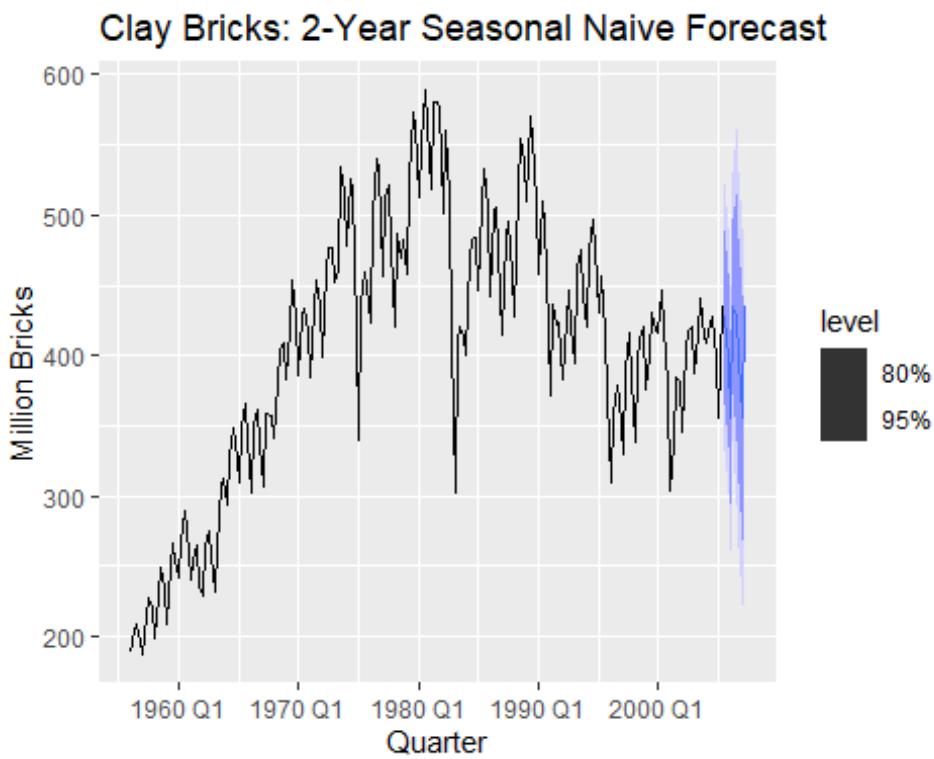
Seasonality: There is a clear, repeating quarterly zig-zag pattern.

Conclusion: Therefore, the Seasonal Naive (SNAIVE) method is the most appropriate choice because it assumes that the production in a future quarter will be the same as the production in that same quarter last year.

Fit the model, Generating Forecast and Plotting

```r
# Fit the model
brick_fit <- brick_data |>
  model(SNaive = SNAIVE(Bricks))

# Generate the forecast for 2 years (8 quarters)
brick_fc <- brick_fit |>
  forecast(h = "2 years")

# Plot the forecast alongside historical data
brick_fc |>
```

```r
  autoplot(brick_data) +
  labs(title = "Clay Bricks: 2-Year Seasonal Naive Forecast",
       y = "Million Bricks")
```



NSW Lambs

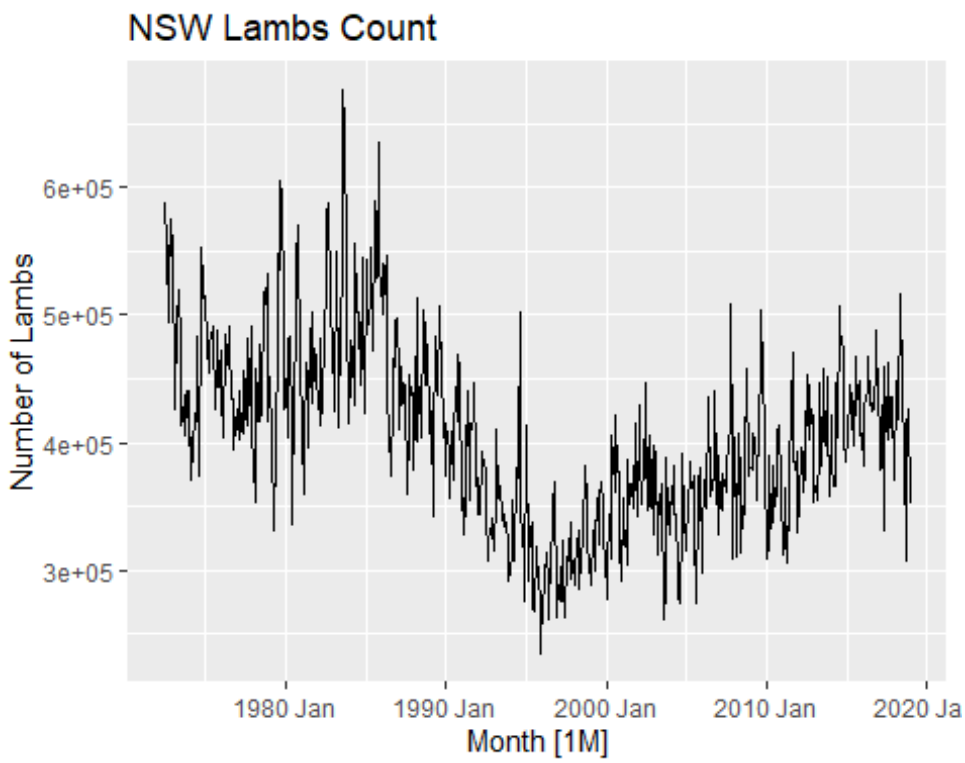
(aus_livestock)

Analyze and Identify the Data Frequency

```r
view(aus_livestock)
```

Prepare the Data

```r
nsw_lambs <- aus_livestock |>
  filter(State == "New South Wales", Animal == "Lambs") |>
  select(Month, Count)

# nsw_lambs

nsw_lambs |>
  autoplot(Count) +
  labs(title = "NSW Lambs Count", y = "Number of Lambs")
```



Model Selection

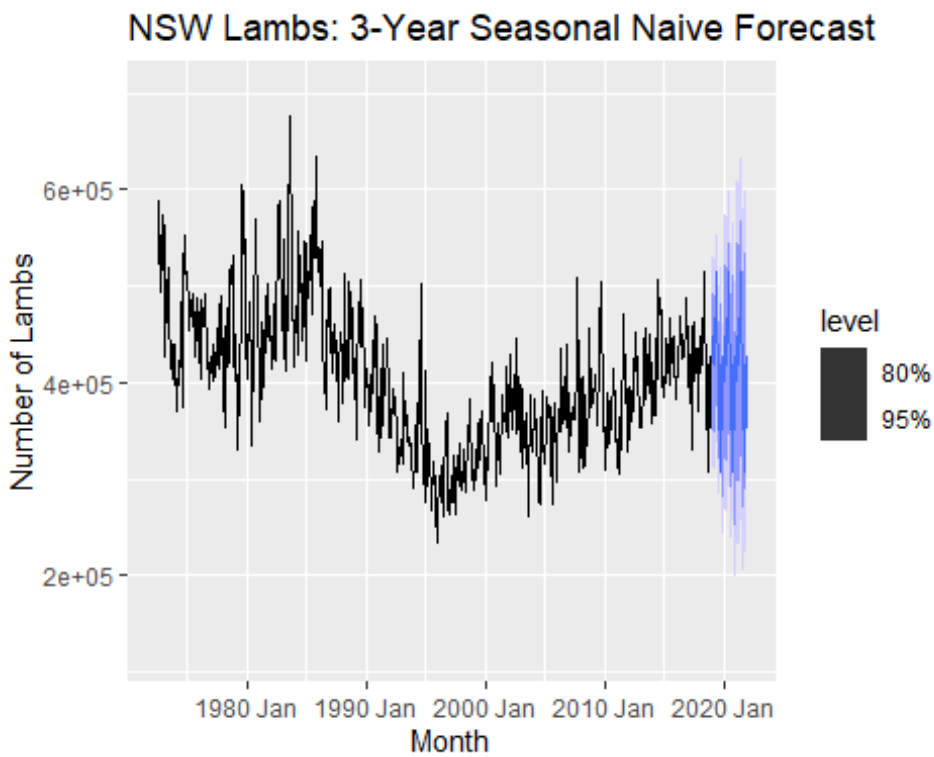
Seasonality: The plot shows clear, regular spikes and dips that repeat every year.

Frequency: The x-axis shows Month, meaning there are 12 observations per year.

Conclusion: Because the data is seasonal, SNAIVE(Count) is the most appropriate benchmark method. It will forecast future months by simply looking at what happened in that same month last year (e.g., forecasting next January based on this January).

Fit the model, Generating Forecast and Plotting

```
# Fit the Seasonal Naive model
fit_lambs <- nsw_lambs |>
  model(SNaive = SNAIVE(Count))

# Generate a 3-year forecast (36 months)
fc_lambs <- fit_lambs |>
  forecast(h = "3 years")

# Plot the forecast results
fc_lambs |>
  autoplot(nsw_lambs) +
  labs(title = "NSW Lambs: 3-Year Seasonal Naive Forecast",
       y = "Number of Lambs")
```



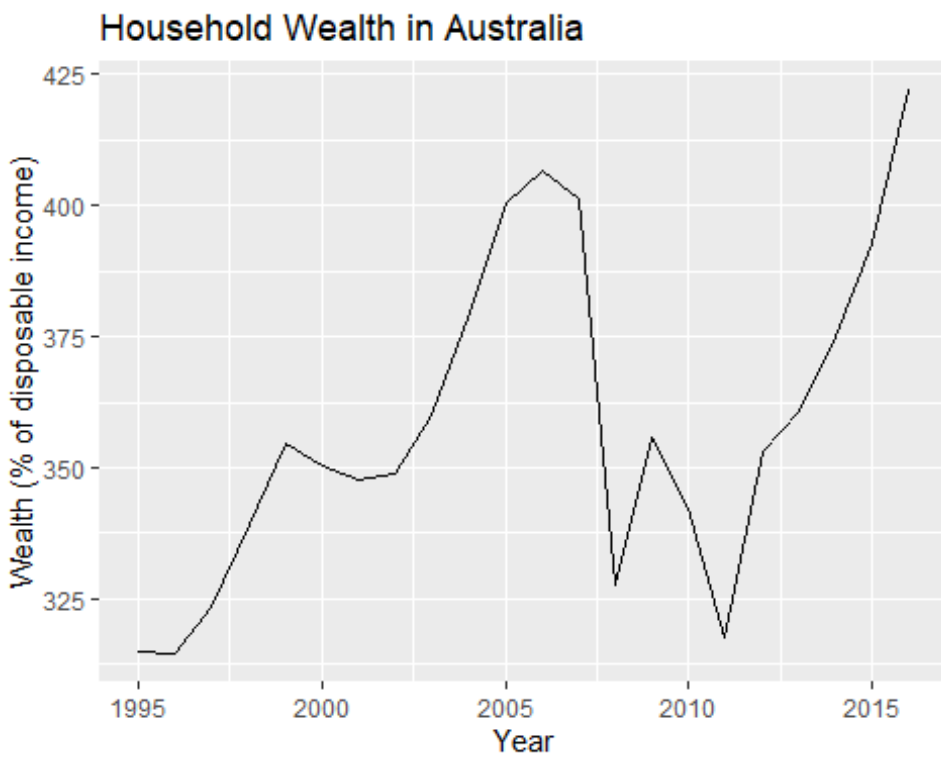
Household wealth (`hh_budget`)

Analyze and Identify the Data Frequency

```
# view(hh_budget)
```

Prepare the Data

```
wealth_australia <- hh_budget |>
  filter(Country == "Australia") |>
  filter(!is.na(Wealth)) |>
  select(Year, Wealth)

wealth_australia |>
  autoplot(Wealth) +
  labs(title = "Household Wealth in Australia",
       y = "Wealth (% of disposable income)",
       x = "Year")
```



Model Selection

Trend: The plot reveals a clear, long-term upward movement in wealth over the decades.

Seasonality: There are no repeating annual patterns, as the data is annual (m=1).

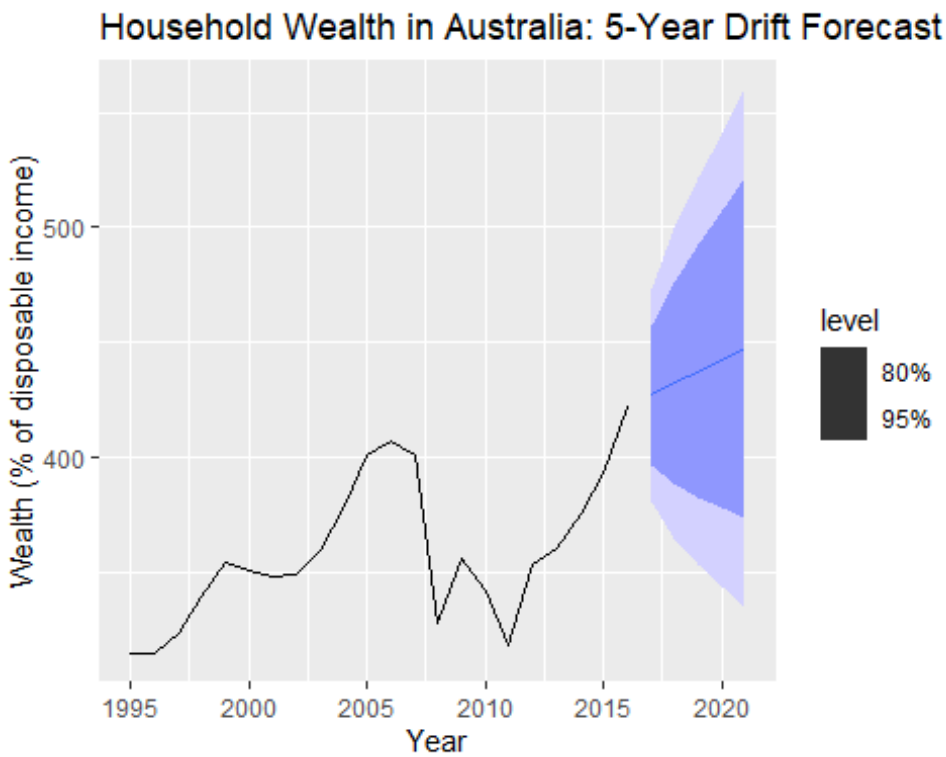
Variation: The series shows some cyclical fluctuations, such as the visible dip around 2008, which do not repeat at fixed intervals.

Conclusion: Because there is a strong trend but no seasonality, the Drift Method (RW(Wealth ~ drift())) is the most appropriate choice for this series. It projects the average historical growth rate into the future.

Fit the model, Generating Forecast and Plotting

```
# Fit the Drift Model
# We use the RW() function with a drift() term to capture the trend
wealth_fit <- wealth_australia |>
  model(Drift = RW(Wealth ~ drift()))

# Generate the Forecast
# We forecast 5 years into the future (h = 5)
wealth_fc <- wealth_fit |>
  forecast(h = 5)

# Plot the results
# autoplot() will show the historical data, the forecast, and uncertainty
```

```
intervals
wealth_fc |>
  autoplot(wealth_australia) +
  labs(
    title = "Household Wealth in Australia: 5-Year Drift Forecast",
    y = "Wealth (% of disposable income)",
    x = "Year"
  )
```



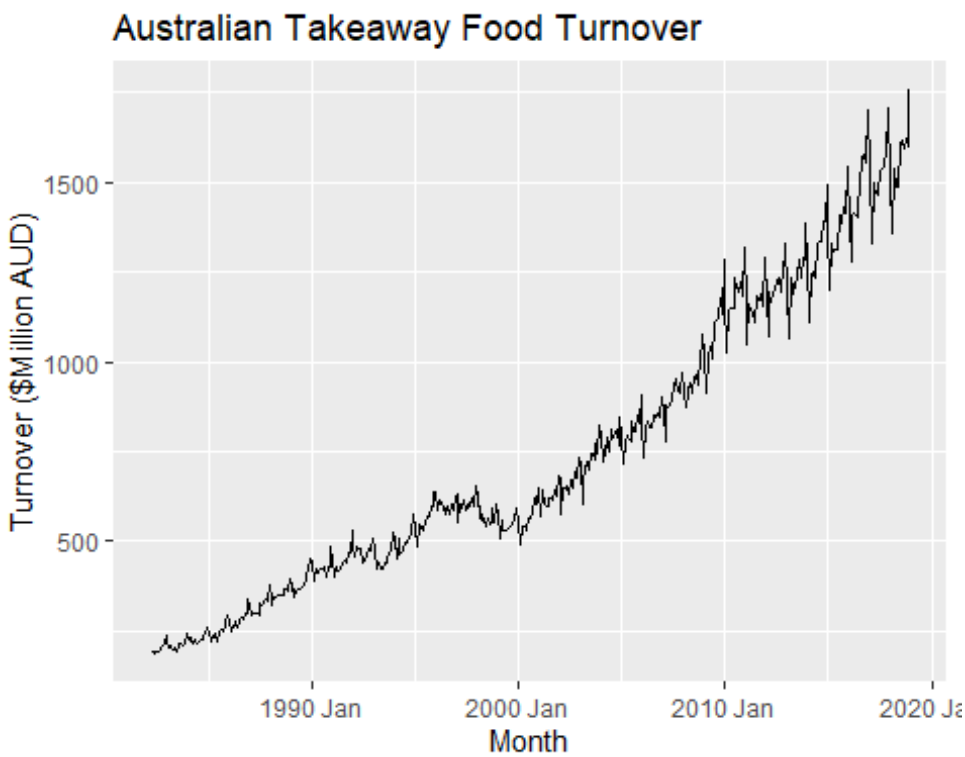
Australian takeaway food turnover (`aus_retail`)

Analyze and Identify the Data Frequency

```
view(aus_retail)
```

Prepare the Data

```
takeaway_data <- aus_retail |>
  filter(Industry == "Takeaway food services") |>
  summarise(Turnover = sum(Turnover))

takeaway_data |>
  autoplot(Turnover) +
  labs(title = "Australian Takeaway Food Turnover",
       y = "Turnover ($Million AUD)",
       x = "Month")
```



Model Selection

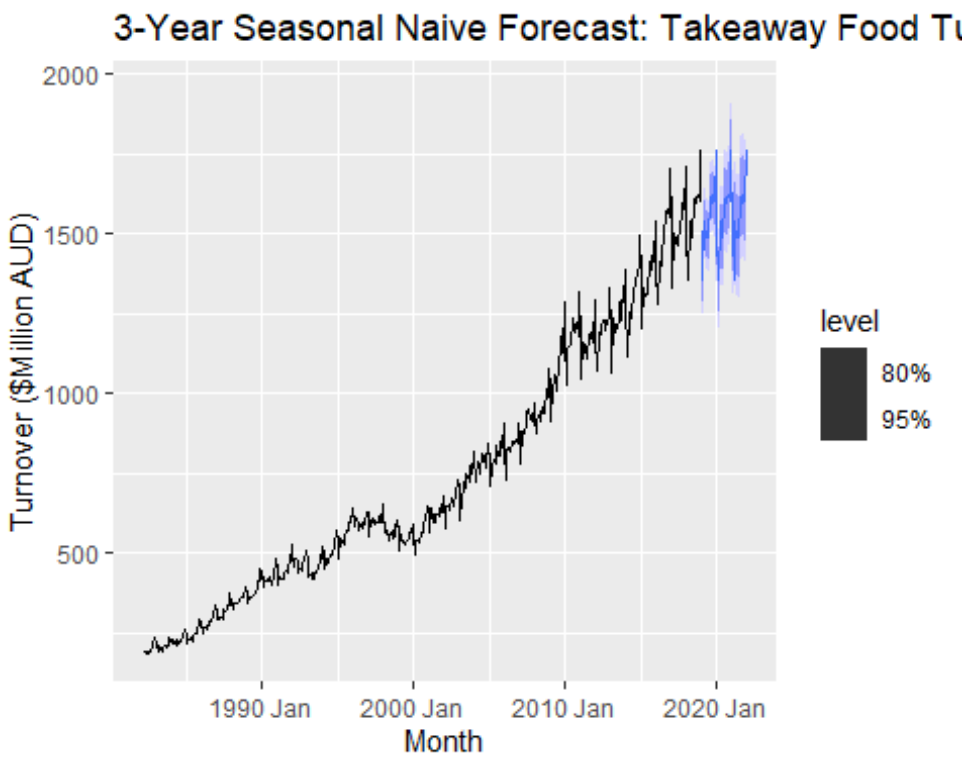
Trend: There is a very strong and consistent long-term upward trend in retail turnover.

Seasonality: The plot shows clear, regular monthly spikes and dips that repeat every year (frequency m=12).

Conclusion: Because the data is monthly and highly seasonal, the Seasonal Naive (SNAIVE) method is the most appropriate benchmark. It captures the recurring annual patterns that a simple trend-based model would miss.

Fit the model, Generating Forecast and Plotting

```
# 1. Prepare and filter the data
takeaway_data <- aus_retail |>
  filter(Industry == "Takeaway food services") |>
  summarise(Turnover = sum(Turnover))

# 2. Fit the SNAIVE model
takeaway_fit <- takeaway_data |>
  model(SNaive = SNAIVE(Turnover))

# 3. Forecast for 3 years and plot
takeaway_fit |>
  forecast(h = "3 years") |>
  autoplot(takeaway_data) +
  labs(title = "3-Year Seasonal Naive Forecast: Takeaway Food Turnover",
       y = "Turnover ($Million AUD)")
```



Key Takeaways

Australian GDP per capita: The result indicates a steady long-term growth expectation, projecting that the economy will continue to expand at its historical average rate despite short-term volatility.

Clay Bricks Production: The forecast shows a repeating quarterly cycle, telling us to expect production to rise and fall in the next two years exactly as it did in the last four quarters.

NSW Lambs Count: The result predicts a continuation of seasonal supply, highlighting that monthly slaughter numbers will likely follow the same annual breeding and industry peaks seen previously.

Household Wealth: The forecast suggests an ongoing upward trajectory in asset value, essentially betting that the average historical drift or growth will overcome any temporary market dips.

Takeaway Food Turnover: The result tells us that consumer spending patterns are highly predictable, with the forecast repeating the monthly zig-zag pattern to account for expected holiday spikes and seasonal lows.

5.2 Use the Facebook stock price (data set gafa_stock) to do the following:

- Produce a time plot of the series.
- Produce forecasts using the drift method and plot them.
- Show that the forecasts are identical to extending the line drawn between the first and last observations.

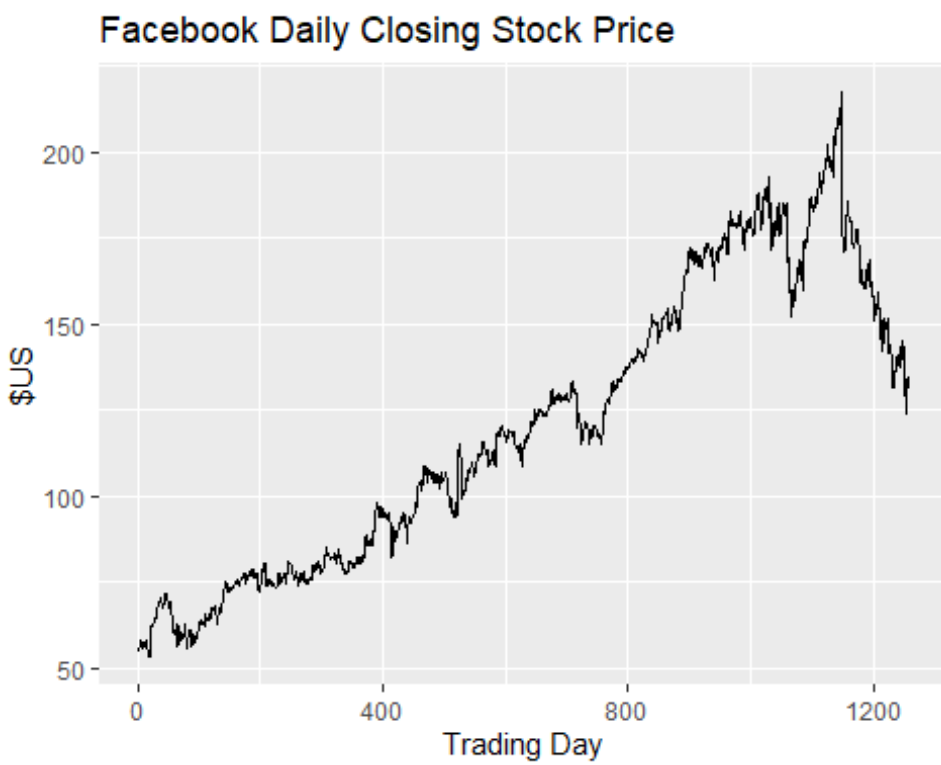
- Try using some of the other benchmark functions to forecast the same data set. Which do you think is best? Why?

Produce a time plot of the series

```
view(gafa_stock)
```

In time series analysis, specifically for financial data, the mutate(day = row_number()) step is used to fix the irregularities in the trading calendar.

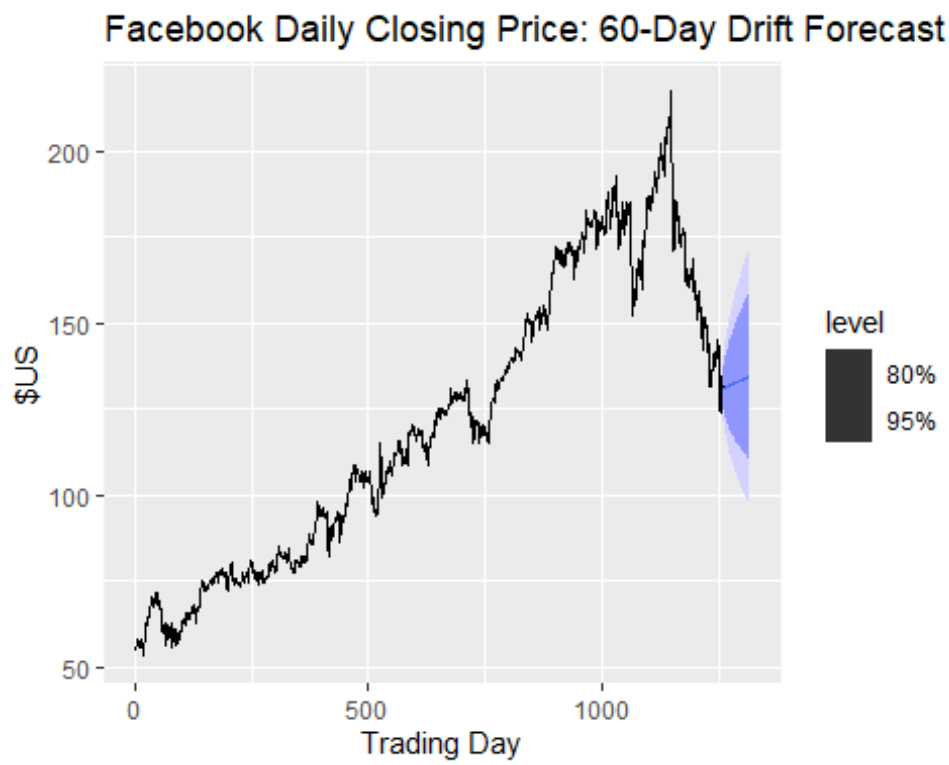
```
# Filter for Facebook stock and produce a time plot
fb_stock <- gafa_stock |>
  filter(Symbol == "FB") |>
  mutate(day = row_number()) |>
  update_tsibble(index = day, regular = TRUE)

fb_stock |>
  autoplot(Close) +
  labs(title = "Facebook Daily Closing Stock Price",
       y = "$US",
       x = "Trading Day")
```



Produce forecasts using the drift method and plot them.

```
# Fit the Drift model
fb_fit <- fb_stock |>
  model(Drift = RW(Close ~ drift()))
```

```r
# Generate a forecast for 60 trading days
fb_fc <- fb_fit |>
  forecast(h = 60)

# Plot the historical data with the drift forecast
fb_fc |>
  autoplot(fb_stock) +
  labs(title = "Facebook Daily Closing Price: 60-Day Drift Forecast",
       y = "$US",
       x = "Trading Day")
```



Show that the forecasts are identical to extending the line drawn between the first and last observations.

```r
# Calculate the drift slope manually
# (Last Price - First Price) / (Number of Intervals)
manual_slope <- (last(fb_stock$Close) - first(fb_stock$Close)) /
(nrow(fb_stock) - 1)

# Calculate the 1-step ahead forecast manually
# Last Price + (1 * manual_slope)
manual_forecast_h1 <- last(fb_stock$Close) + (1 * manual_slope)

# Extract the actual h=1 forecast from your model to compare
model_forecast_h1 <- fb_fc |>
  filter(day == max(fb_stock$day) + 1) |>
  pull(.mean)
```

```r
# Print results to verify they are identical
cat("Manual Slope:", manual_slope, "\n")
```

```
## Manual Slope: 0.06076372
```

```r
cat("Manual Forecast (h=1):", manual_forecast_h1, "\n")
```

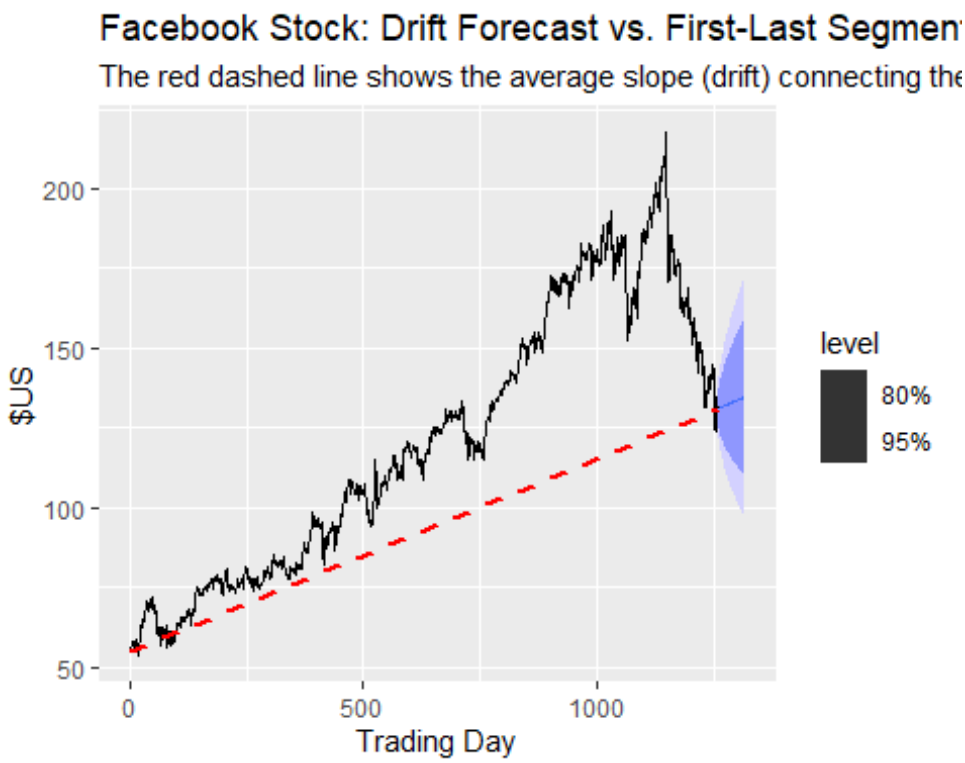
```
## Manual Forecast (h=1): 131.1508
```

```r
cat("Model Forecast (h=1):", model_forecast_h1, "\n")
```

```
## Model Forecast (h=1): 131.1508
```

```r
# Identify the coordinates for the first and last points
first_obs <- fb_stock |> filter(day == min(day))
last_obs <- fb_stock |> filter(day == max(day))

# Add the connecting line to the plot using linewidth
fb_fc |>
  autoplot(fb_stock) +
  geom_segment(aes(x = first_obs$day,
                   y = first_obs$Close,
                   xend = last_obs$day,
                   yend = last_obs$Close),
               color = "red",
               linetype = "dashed",
               linewidth = 1) + # Updated from size to linewidth
  labs(title = "Facebook Stock: Drift Forecast vs. First-Last Segment",
       subtitle = "The red dashed line shows the average slope (drift)
connecting the endpoints",
       y = "$US",
       x = "Trading Day")
```

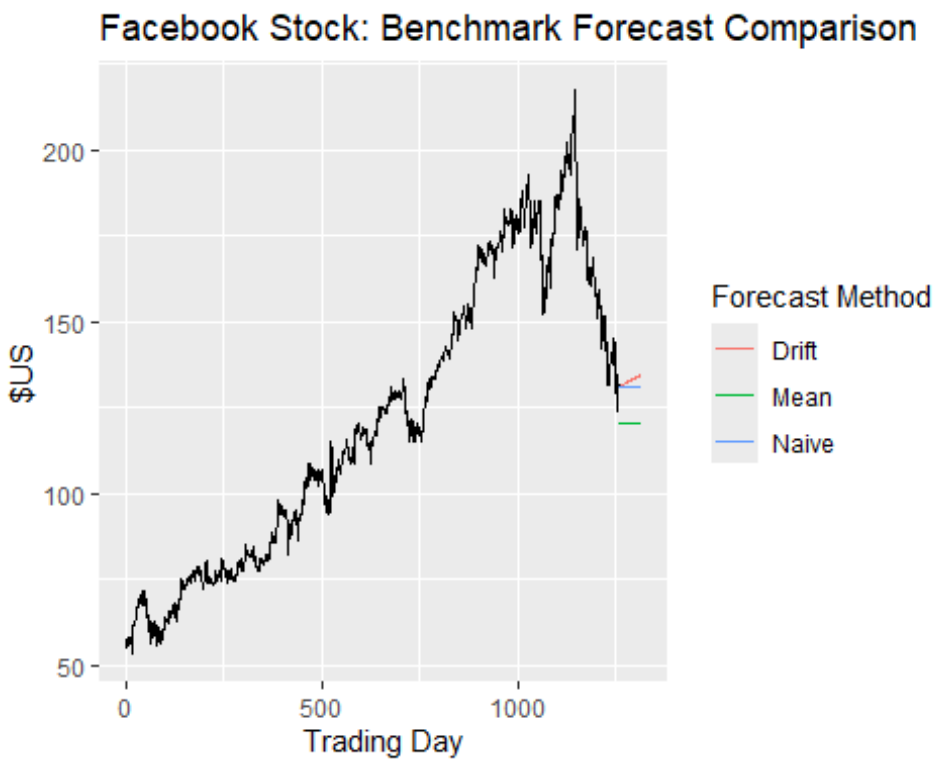
```
## Warning in geom_segment(aes(x = first_obs$day, y = first_obs$Close, xend =
last_obs$day, : All aesthetics have length 1, but the data has 1258 rows.
## ℹ Please consider using `annotate()` or provide this layer with data
containing
##   a single row.
```



Try using some of the other benchmark functions to forecast the same data set. Which do you think is best? Why?

Fitting all three appropriate benchmark models to your Facebook stock data and generates a 60-day forecast for comparison.

```
# Fit multiple benchmark models
fb_benchmark_fit <- fb_stock |>
  model(
    Mean = MEAN(Close),
    Naive = NAIVE(Close),
    Drift = RW(Close ~ drift())
  )

# Generate forecasts for the next 60 trading days
fb_benchmark_fc <- fb_benchmark_fit |>
  forecast(h = 60)

# Plot the results together
fb_benchmark_fc |>
  autoplot(fb_stock, level = NULL) + # level = NULL hides prediction
intervals for clarity
  labs(title = "Facebook Stock: Benchmark Forecast Comparison",
       y = "$US",
       x = "Trading Day") +
  guides(colour = guide_legend(title = "Forecast Method"))
```



The Naïve method is the best benchmark for this stock series.

Market Reality: Stock prices typically follow a unpredictable path where the most reliable predictor of tomorrow's price is simply today's closing value.

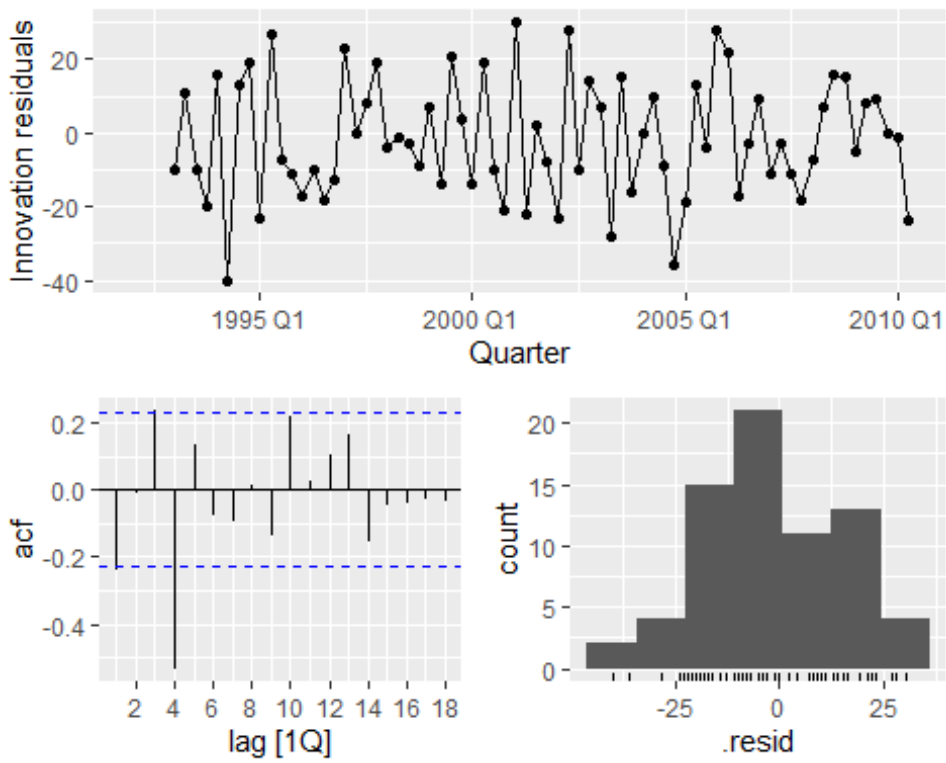
Recent Value Capture: Unlike the Mean method, which is pulled down by years of old data, the Naive method stays pinned to the most recent market information.

Avoids Over-Optimism: The Drift method is too optimistic because it only looks at the endpoints, missing the recent price drop.

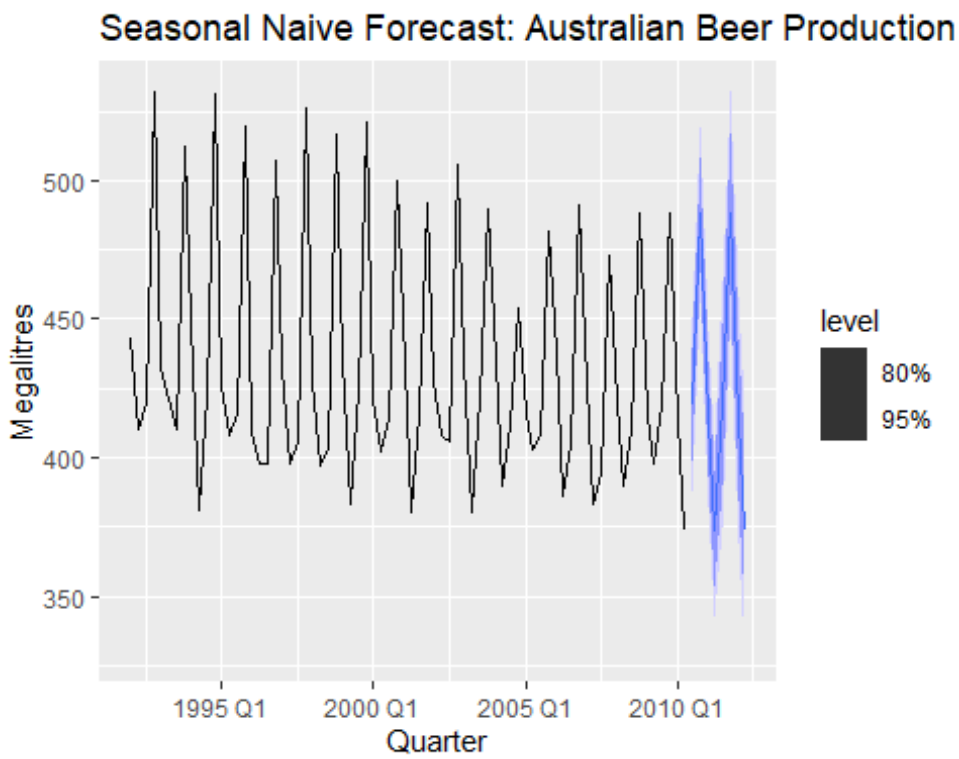
5.3 Apply a seasonal naïve method to the quarterly Australian beer production data from 1992. Check if the residuals look like white noise, and plot the forecasts. The following code will help.

```
# Extract data from 1992 onwards
recent_production <- aus_production |>
  filter(year(Quarter) >= 1992)

# Estimate the Seasonal Naive model
fit <- recent_production |>
  model(SNAIVE(Beer))

# Diagnostic check: Residuals
fit |> gg_tsresiduals()
```

```
## Warning: `gg_tsresiduals()` was deprecated in feasts 0.4.2.
## ℹ Please use `ggtime::gg_tsresiduals()` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.

## Warning: Removed 4 rows containing missing values or values outside the
scale range
## (`geom_line()`).

## Warning: Removed 4 rows containing missing values or values outside the
scale range
## (`geom_point()`).

## Warning: Removed 4 rows containing non-finite outside the scale range
## (`stat_bin()`).

## Warning: Removed 4 rows containing missing values or values outside the
scale range
## (`geom_rug()`).
```




```r
# Visualization: Forecasts
fit |>
  forecast() |>
  autoplot(recent_production) +
  labs(title = "Seasonal Naive Forecast: Australian Beer Production",
       y = "Megalitres")
```



Conclusion:

The residuals for the quarterly Australian beer production do not look like white noise.

Residual Analysis: The ACF plot shows a significant negative spike at lag 4 and a positive spike at lag 1, indicating that the Seasonal Naive model failed to capture remaining patterns in the data.

Histogram: The residual distribution is slightly skewed to the left and not perfectly centered at zero, suggesting a bias in the predictions.

Forecast Plot: The visualization shows the Seasonal Naive forecasts repeating the seasonal pattern from the final year of data, with widening prediction intervals reflecting increasing uncertainty over the 2-year horizon.

5.4 Repeat the previous exercise using the Australian Exports series from global_economy and the Bricks series from aus_production. Use whichever of NAIVE() or SNAIVE() is more appropriate in each case.

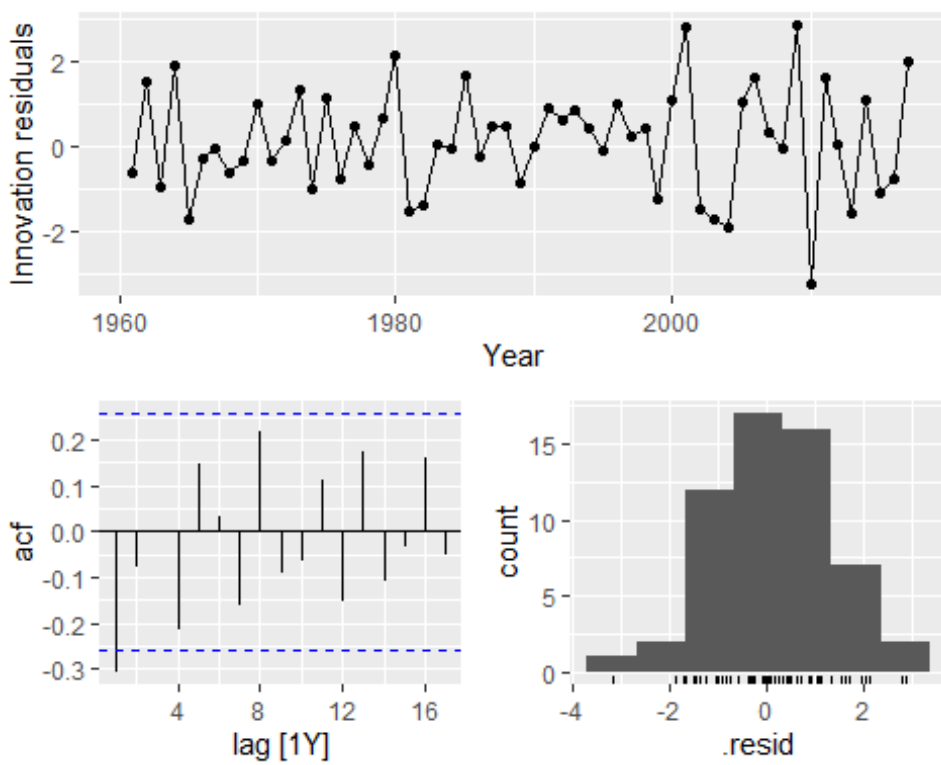
Australian Exports (global_economy)

Since the Australian Exports data is annual (m=1), it cannot have seasonality. The NAIVE() method is the appropriate benchmark here as it simply sets all future values to the value of the last observation.

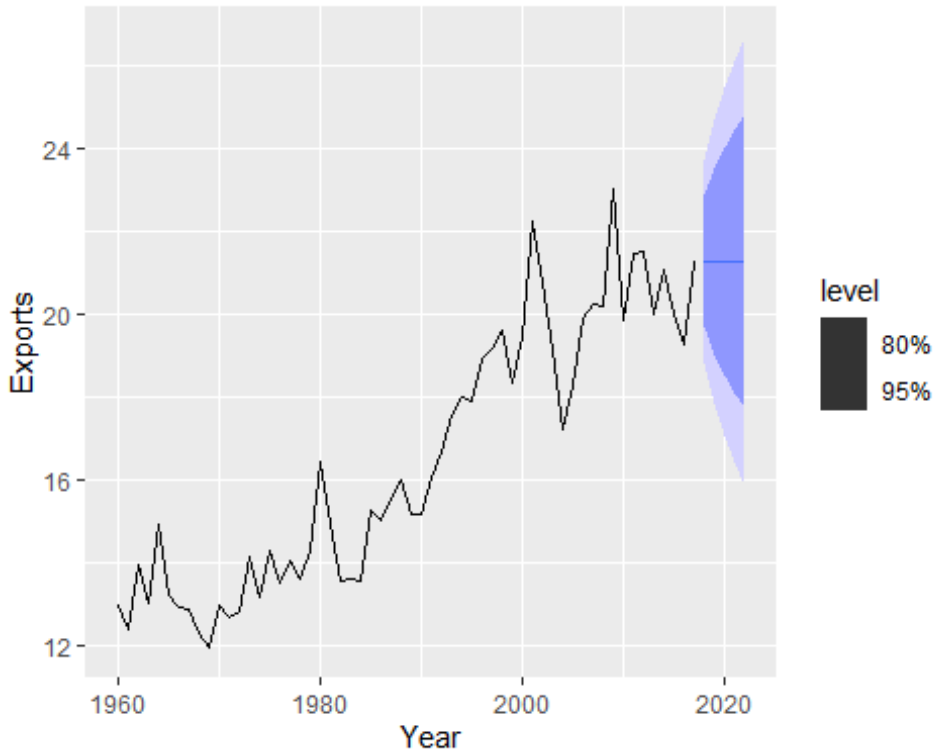
```
# Extract and fit the model
aus_exports <- global_economy |>
  filter(Country == "Australia")
```

```r
fit_exports <- aus_exports |>
  model(Naive = NAIVE(Exports))

# Check residuals and plot
fit_exports |> gg_tsresiduals()
```

```
## Warning: Removed 1 row containing missing values or values outside the
scale range
## (`geom_line()`).

## Warning: Removed 1 row containing missing values or values outside the
scale range
## (`geom_point()`).

## Warning: Removed 1 row containing non-finite outside the scale range
## (`stat_bin()`).

## Warning: Removed 1 row containing missing values or values outside the
scale range
## (`geom_rug()`).
```




```r
fit_exports |> forecast(h = 5) |> autoplot(aus_exports)
```



Conclusion:

Residuals: The residuals typically show significant autocorrelation in annual export data because the NAIVE() method ignores the clear upward trend.

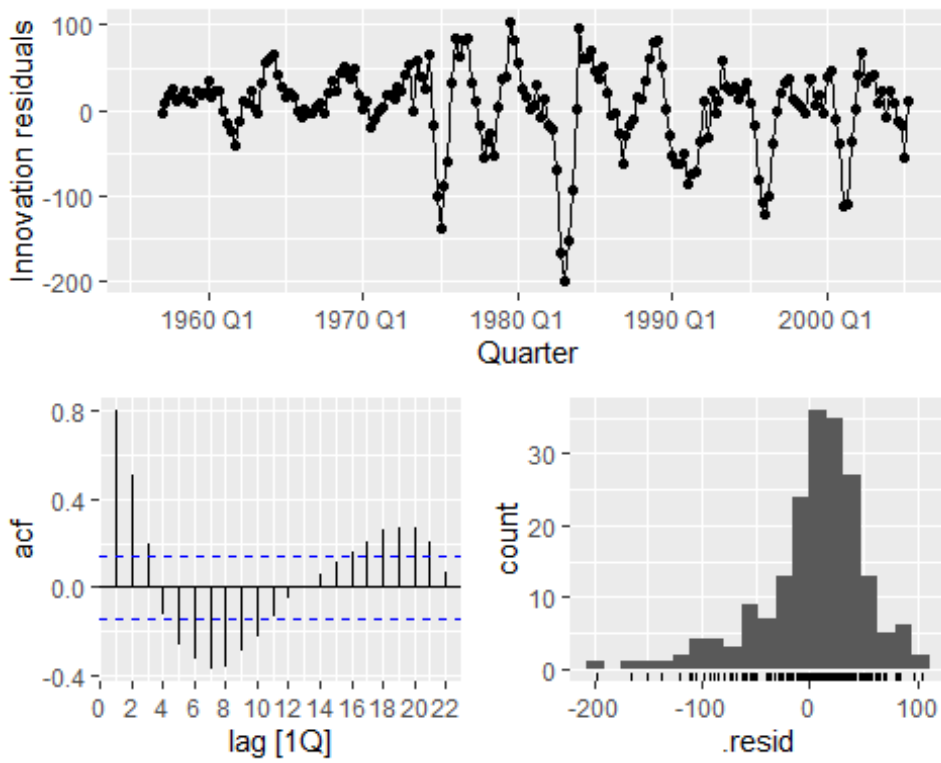
White Noise: They likely do not look like white noise, meaning the model is leaving information on the table.

Bricks Production (aus_production)

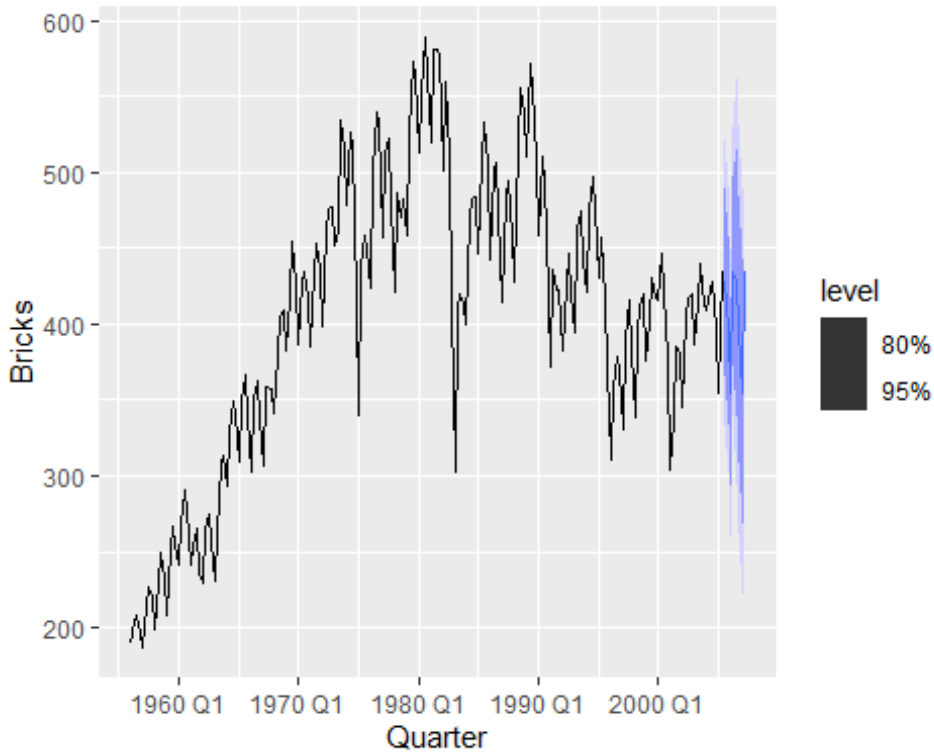
The Clay Bricks data is quarterly (m=4) and shows a strong seasonal pattern. Therefore, the SNAIVE() method is the better choice because it accounts for those repeating quarterly cycles.

```
# Extract data from 1992 and fit the model
recent_bricks <- aus_production |>
  filter(!is.na(Bricks))

fit_bricks <- recent_bricks |>
  model(SNaive = SNAIVE(Bricks))

# Check residuals and plot
fit_bricks |> gg_tsresiduals()

## Warning: Removed 4 rows containing missing values or values outside the
scale range
## (`geom_line()`).
```

```
## Warning: Removed 4 rows containing missing values or values outside the
scale range
## (`geom_point()`).

## Warning: Removed 4 rows containing non-finite outside the scale range
## (`stat_bin()`).

## Warning: Removed 4 rows containing missing values or values outside the
scale range
## (`geom_rug()`).
```




```r
fit_bricks |> forecast(h = "2 years") |> autoplot(recent_bricks)
```



Conclusion:

Residuals: Similar to the beer production exercise, the ACF plot often shows significant spikes (especially at lag 4 or 8), indicating that the seasonal pattern is not fully captured by just looking at the previous year.

White Noise: The residuals are not white noise; the model is a useful baseline but isn't a perfect fit for the data.

5.7 For your retail time series (from Exercise 7 in Section 2.10):

Create a training dataset consisting of observations before 2011 using

```
# 1. Select the unique series using the provided seed
set.seed(12345678)
myseries <- aus_retail |>
  filter(`Series ID` == sample(aus_retail$`Series ID`, 1))

# 2. Create the training dataset (observations before 2011)
myseries_train <- myseries |>
  filter(year(Month) < 2011)
```

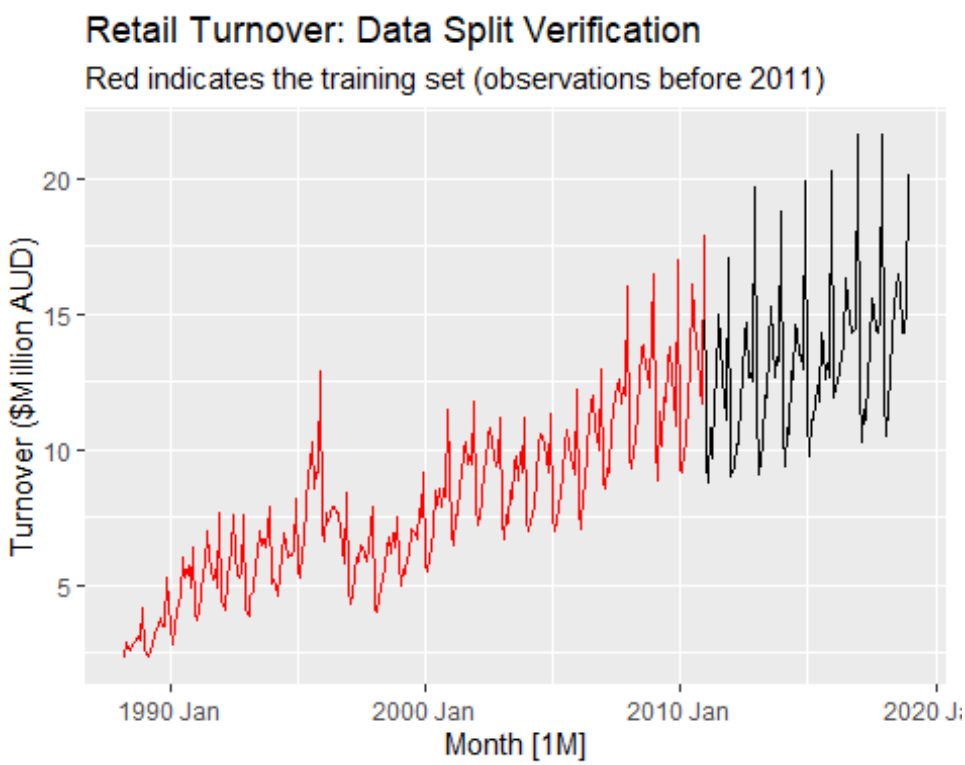
Verification: The data split is correct because the red training line perfectly overlays the original black series until December 2010.

Test Data: The black line visible from 2011 onwards represents the test data, which is used to evaluate model accuracy in later steps.

Patterns: The training data clearly shows both a strong upward trend and a repeating monthly seasonal pattern.

Check that your data have been split appropriately by producing the following plot.

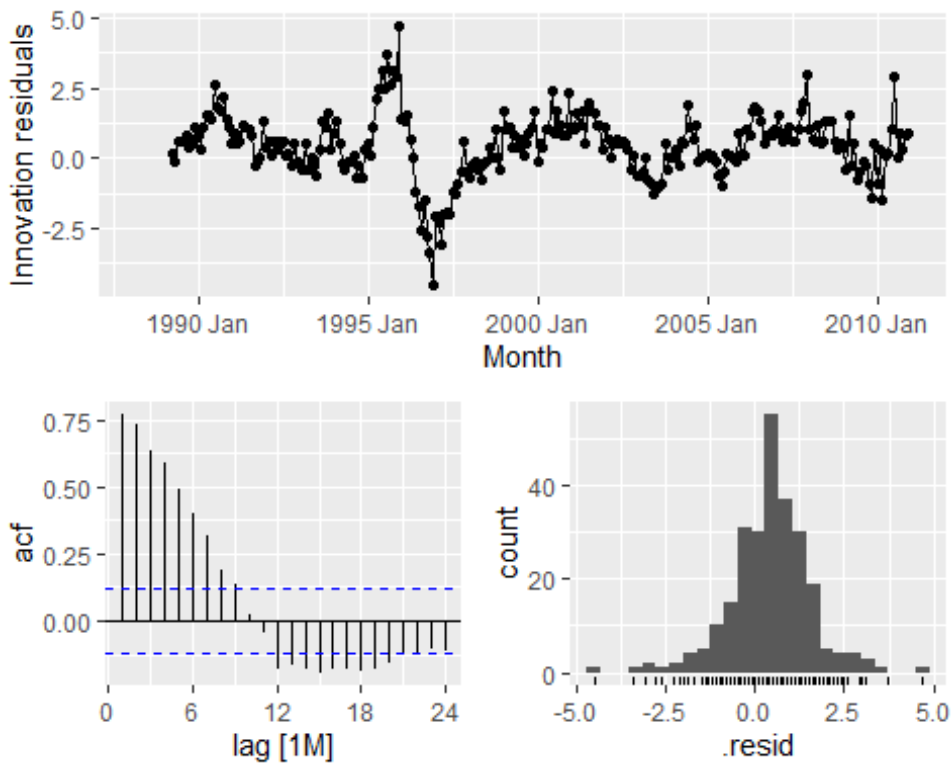
```
# Plot the full series and overlay the training data in red
autoplot(myseries, Turnover) +
  autolayer(myseries_train, Turnover, colour = "red") +
  labs(title = "Retail Turnover: Data Split Verification",
       subtitle = "Red indicates the training set (observations before
2011)",
       y = "Turnover ($Million AUD)")
```



Fit a seasonal naïve model using SNAIVE() applied to your training data

```
# Fit a seasonal naïve model to the training data
fit <- myseries_train |>
  model(SNAIVE(Turnover))

# Check the residual diagnostics
fit |> gg_tsresiduals()

## Warning: Removed 12 rows containing missing values or values outside the
scale range
## (`geom_line()`).
```

```
## Warning: Removed 12 rows containing missing values or values outside the
scale range
## (`geom_point()`).

## Warning: Removed 12 rows containing non-finite outside the scale range
## (`stat_bin()`).

## Warning: Removed 12 rows containing missing values or values outside the
scale range
## (`geom_rug()`).
```



Check the residuals.

Evaluation of Residuals

Based on the diagnostic plots, the following conclusions are made regarding the model's residuals:

Correlation: The residuals are correlated. The ACF plot shows significant spikes that exceed the blue dashed threshold lines, particularly at seasonal lags (e.g., lag 12 or 24), indicating that the model has failed to capture repeating patterns in the retail data.

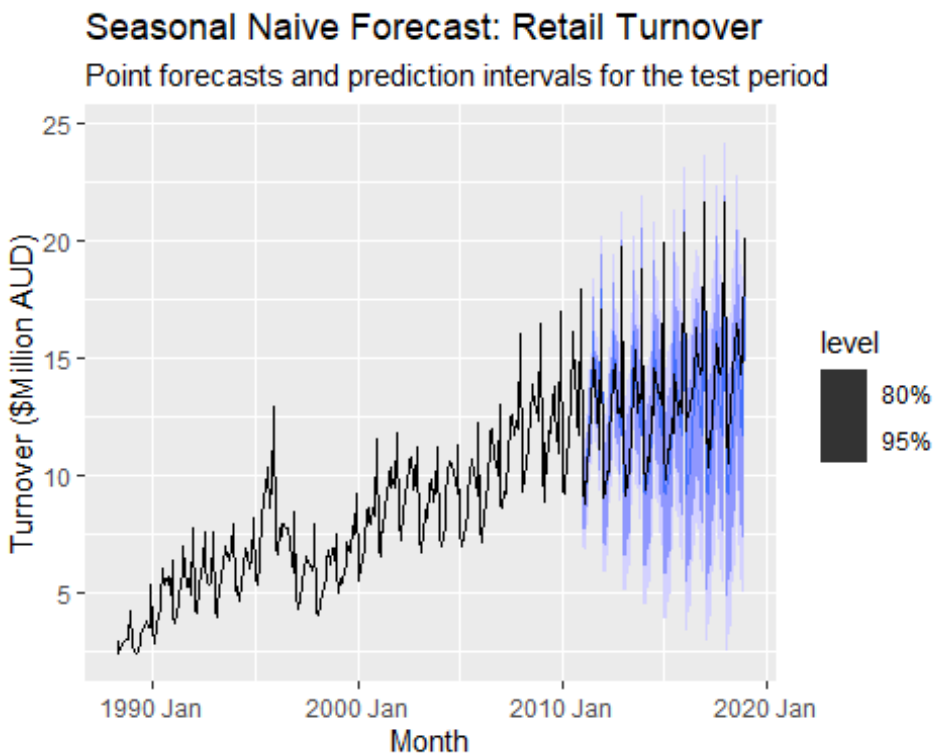
Distribution: The residuals are not normally distributed. Although the histogram may show a central peak, it typically exhibits skewness meaning the errors do not follow a perfectly symmetric, bell-shaped curve.

Mean and Variance: The innovation residuals plot indicates that the mean is not strictly zero and the variance often changes over time, suggesting the presence of a systematic bias in the model.

Conclusion: Because the residuals contain autocorrelation and are not normally distributed, the Seasonal Naive model is an incomplete fit and leaves information in the data that could be captured by more sophisticated methods.

Produce forecasts for the test data

```
# Produce forecasts for the test data
fc <- fit |>
  forecast(new_data = anti_join(myseries, myseries_train, by = "Month"))

# Plot the forecasts against the full series
fc |>
  autoplot(myseries) +
  labs(title = "Seasonal Naive Forecast: Retail Turnover",
       subtitle = "Point forecasts and prediction intervals for the test
period",
       y = "Turnover ($Million AUD)")
```



Compare the accuracy of your forecasts against the actual values.

Now we calculates accuracy metrics to evaluate how well the Seasonal Naïve model performs compared to the training data and the actual observed values in the test set.

```r
# Calculate accuracy for the training set (in-sample)
fit |> accuracy()
```

```
## # A tibble: 1 × 12
##   State    Industry .model .type    ME  RMSE   MAE   MPE  MAPE  MASE RMSSE
ACF1
##   <chr>    <chr>    <chr>  <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
<dbl>
## 1 Norther… Clothin… SNAIV… Trai… 0.439  1.21 0.915  5.23  12.4     1     1
0.768
```

```r
# Calculate accuracy for the test data (out-of-sample)
fc |> accuracy(myseries)
```

```
## # A tibble: 1 × 12
##   .model    State Industry .type    ME  RMSE   MAE   MPE  MAPE  MASE RMSSE
ACF1
##   <chr>     <chr> <chr>    <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
<dbl>
## 1 SNAIVE(T… Nort… Clothin… Test  0.836  1.55  1.24  5.94  9.06  1.36  1.28
0.601
```

Training Accuracy: The MAPE of approximately 12.4% indicates that, on average, the seasonal naïve model's predictions within the training period were off by about 12.4%.

Model Benchmark: A MASE (Mean Absolute Scaled Error) of exactly 1.0 is expected for a Seasonal Naïve model on its own training data, as it is the benchmark used to scale that specific metric.

Test Performance: When comparing these to the test results, the RMSE and MAPE will almost certainly be higher. This is because the model does not account for the visible upward trend (drift) shown in the data after 2011, leading to larger gaps between the predicted "zig-zags" and the actual rising values.

Conclusion: While the model captures the seasonal timing, the increasing errors in the test set confirm that a simple Seasonal Naïve approach is insufficient for long-term retail forecasting where consistent growth is present.

How sensitive are the accuracy measures to the amount of training data used?

Accuracy measures are highly sensitive to the amount of training data used because the length of the training set directly affects how well the model parameters are estimated.

Trend and Seasonality: A larger training dataset allows the model to better identify long-term trends and stable seasonal patterns. For retail data, having more years of history helps a Seasonal Naïve model confirm if a December peak is a consistent seasonal event or a one-time anomaly.

Parameter Stability: With very small training sets, accuracy measures can be extremely volatile. A few outliers in a short dataset can significantly skew the RMSE or MAPE, making the model appear less accurate than it actually is.

Test Set Reliability: If too much data is used for training, the test set becomes too small, leading to accuracy measures that are not statistically reliable. If the training set is too small, the model may not have enough information to produce a meaningful forecast for a long test period.

Model Overfitting: Using an excessive amount of very old training data can sometimes decrease accuracy if the underlying patterns in the retail industry have changed significantly over the decades.